

Layer 03: Core (Silver)

Script: `sql/03_core.sql` | Status: Complete | Next layer: Marts (`04_marts.sql`)

1. Purpose

Turn the untouched staging tables into a clean, deduplicated, correctly typed star schema. Every decision here was decided in advance by evidence from profiling (`docs/02_profiling_findings.md`), not chosen while writing this script.

2. What this layer produces

Table	Grain
<code>dim_calendar</code>	One row per calendar day, 2022-01-01 to 2024-12-31
<code>dim_provider</code>	One row per provider
<code>dim_member</code>	One row per real member, plus one shared "Unknown" row
<code>fact_claim</code>	One row per real, deduplicated claim, in its current (post-adjustment) state

3. Design decisions, and the profiling evidence behind each

Decision	What it does	Evidence
D1/D2	<code>claim_id</code> is not the key. Duplicates (14,775 pairs) are resolved by scoring each copy on five validity checks and keeping the best, tie-broken by the earliest <code>row_id</code>	2.1, 2.2 found duplicates that often disagree, not identical copies
D5	Orphan claims (<code>member_id</code> not in membership at all) point to one shared <code>UNKNOWN</code> row in <code>dim_member</code> , not an invented row each	3.1: 2.55% of claims are orphans
D6	<code>outside_enrollment</code> flags a claim with no membership row for that member in that month. This also catches every orphan, since an orphan by definition has no membership row either	3.3
D7/D8	"Not applicable" blanks (length of stay on a pharmacy claim) stay <code>NULL</code> . A genuinely missing <code>procedure_code</code> stays <code>NULL</code> plus a <code>procedure_code_missing</code> flag	4.1, 4.2
D9	Ages outside 0-110 become <code>NULL</code> plus an <code>age_invalid</code> flag, never guessed or deleted	5.1
D10	<code>paid_amount</code> is capped at <code>allowed_amount</code> where it exceeds it; the original is kept in <code>original_paid_amount</code>	5.2
D11	<code>load_timestamp</code> is parsed in two valid formats, not one	5.4 (F8)
D12	One mapping standardizes all nine raw <code>claim_status</code> spellings to Paid/Denied/Pending	6.1
D13	<code>place_of_service</code> is dropped. It was a perfect 1-to-1 copy of <code>claim_type</code> with zero exceptions	6.2 (F10)
D15	An adjustment record replaces its original's <code>paid_amount</code> . The pre-adjustment value is kept for audit	7.1, 7.2
D16	Member and provider attributes (sex, chronic flags, region, plan, type, specialty, network status) are stored once in the dimension, not repeated on every claim	8.1, 8.2, 8.3: zero conflicts across 3,000 providers and 75,000 members

4. What was deliberately left out of `fact_claim`, and why

Six columns from staging do not appear on the fact table at all, because profiling proved they belong in a dimension instead:

- `region`, `patient_sex` → stable per member (8.2/8.3), now live only in `dim_member`
- `provider_type`, `provider_specialty`, `network_status` → stable per provider (8.1), now live only in `dim_provider`
- `place_of_service` → proven redundant with `claim_type` (D13)

`patient_age` stays on `fact_claim`, deliberately. Unlike the columns above, it is not a property of the member, it is a property of this specific claim (the member's age at the time of that visit), so it correctly varies row to row.

One honest gap: provider's own region exists inside the data generator but was never written into `claims_raw.csv`. `dim_provider` has no region column, because inventing one would mean claiming data the pipeline doesn't actually have.

5. Issues hit while building this layer

#	Symptom	Root cause	Fix	Lesson
1	Error 1064 creating <code>dim_calendar</code> , near <code>year_month</code>	<code>YEAR_MONTH</code> is a reserved word in MySQL (used in <code>INTERVAL ... YEAR_MONTH</code>) -- the same mistake as the staging layer, repeated	Renamed the column to <code>month_year</code>	Check column names against reserved words before building, not after
2	Error 1411: <code>Incorrect datetime value: '2025-13-45'</code>	The format check only validated digit <i>shape</i> (4 digits, dash, 2 digits, dash, 2 digits), not real calendar ranges. <code>2025-13-45</code> matches that shape despite month 13 and day 45 not existing, so <code>STR_TO_DATE</code> was asked to parse an impossible date and hard-errored instead of returning NULL	Rewrote the check to validate month 01-12 and day 01-31 before attempting to parse	A value can be shape-valid and still calendar-invalid. Validate ranges, not just format, before casting
3	The dedup step ran for 18+ minutes before being cancelled	<code>ROW_NUMBER() OVER (...)</code> forces a full sort of the ~1M-row, ~20-column temporary table. MySQL's temp-table memory ceiling (<code>(max_heap_table_size)</code> , default 16MB) was far smaller than the sort needed, so it spilled to disk	Replaced the window-function sort with a three-step <code>GROUP BY</code> -based dedup (max score per claim, then earliest <code>row_id</code> among ties, then recover the full row), which never needs to sort the wide table. Also raised <code>(tmp_table_size)/(max_heap_table_size)</code> for future large builds	A window function that requires sorting a wide table is often far more expensive than an equivalent <code>GROUP BY</code> plus a

#	Symptom	Root cause	Fix	Lesson
				join, even though they return the same answer

6. Reconciliation (proof nothing was silently lost or duplicated)

Check	Result
Distinct original claim IDs in staging vs. rows in <code>(fact_claim)</code> vs. distinct claim IDs in <code>(fact_claim)</code>	985,000 / 985,000 / 985,000 -- exact match
Distinct members in membership vs. real rows in <code>(dim_member)</code>	49,975 / 49,975 -- exact match
Distinct providers in staging vs. rows in <code>(dim_provider)</code>	3,000 / 3,000 -- exact match
<code>(unmatched_member)</code> rate	2.52% (vs. 2.55% in staging -- the small drop is expected: the dedup scoring rewards a real member match, so it slightly favors keeping matched copies over orphaned ones)
<code>(outside_enrollment)</code> rate	5.92% (not directly comparable to profiling's 3.39% -- this flag also marks every orphan, which profiling's narrower query excluded; the two numbers reconcile once that difference is accounted for)
Rows with an adjustment applied	14,775 -- matches profiling's "distinct originals adjusted" exactly
Rows capped at <code>(allowed_amount)</code>	8,117
Rows with an invalid age	2,914
Rows missing <code>(procedure_code)</code>	43,740 (4.44% of 985,000 -- consistent with the ~4.5% designed rate)
Total paid: staging originals vs. <code>(fact_claim)</code>	\$3,013,037,705 vs. \$2,935,155,860 -- lower is expected and correct, since dedup removes duplicate dollars and capping removes erroneous inflated amounts

Dollar totals are reported for context, not as a pass/fail check -- a lower total after cleaning is the point of this layer, not a sign of data loss.

7. What this layer deliberately does not do

- Coverage-gap filling (D4) is still deferred to the marts layer, where the exposure table is built and the gap-run-length evidence will decide the rule

- No feature engineering (point-in-time prior-claims counts, PMPM, the lag triangle) -- that is the marts layer's job
- No frequency or severity modeling -- that is the Python analysis layer

8. Interview one-liners

- **30 sec:** "The core layer turned a dirty, duplicated staging table into a clean star schema, with every cleaning rule traced back to a specific number from profiling, not a guess."
- **On the dedup bug:** "A window-function sort on a million-row table was spilling to disk and taking 18 minutes. I replaced it with an equivalent GROUP BY-based approach that gets the identical answer without sorting the whole table, and proved it was identical with a reconciliation query before trusting it."
- **On the date bug:** "A malformed value can look format-valid and still be calendar-invalid, like a month of 13. I learned to validate real ranges, not just shape, before handing a string to a date parser."